

Reinforcement Learning Algorithms Applied to a Pacman Grid

Q-learning, SARSA, MCTS, and DQN

SINGARIN-SOLE Livio

Jean Monnet University, St-Etienne, France

October 2025

Contents

1	Introduction	2
2	Environment	2
3	Exploration Policy: ε-Greedy	3
4	Tabular Methods: Q-learning vs. SARSA	3
5	Monte Carlo Tree Search (MCTS)	6
6	Deep Q-Network (DQN)	7
7	Robustness to Noise	9
8	Stochastic Environment	10
9	Global Performance Overview	11
10	Conclusion	11

1 Introduction

As a matter of fact, computers now outperform humans in many tasks, particularly in games. This report explores the efficiency of several well-known Reinforcement Learning (RL) methods applied to a simplified version of the PAC-MAN game. The rules are straightforward: you control a small yellow ball in a grid world, and your objective is to eat the pac-dot to win. If you touch a ghost, you lose. The goal of this work is to implement and analyze different algorithms that enable the computer to learn an optimal behavior in such an environment.

2 Environment

In Reinforcement Learning, particularly in Q-learning-based-algorithms, the environment is defined by a Markov Decision Process (MDP) (S, A, P, R, γ) (state, actions, transition probability, reward). The objectives are to design a pacman environment grid, either deterministic or stochastic. States are grid-encoded observations, and actions are the four moves $\{\uparrow, \rightarrow, \downarrow, \leftarrow\}$: a penalty per step, a positive terminal reward for reaching the dot, and a negative terminal penalty for ghost collision.

Rewards. Step cost $c_{\text{step}} = -1$, dot reward $r_{\text{dot}} = +10$, ghost penalty $r_{\text{ghost}} = -10$ (unless stated otherwise). Episodes are truncated at a maximum of T_{max} steps, which is set relatively to the grid size to bound computation ($\approx H \times W$).

Grid. A 3D-matrix (C, H, W) with binary channels for empty cells, walls, dot, ghosts, and Pacman is implemented. During this project, a 10×10 grid is used.

Initial deterministic setup. To isolate learning effects, action noise and ghost dynamics are first disabled : noise probability $p_n = 0$, moving ghost probability $p_g = 0$, no random start. Later sections introduce noise and dynamics. In this type of environment, the transition probability satisfies $P(s' | s, a) \in \{0, 1\}$.

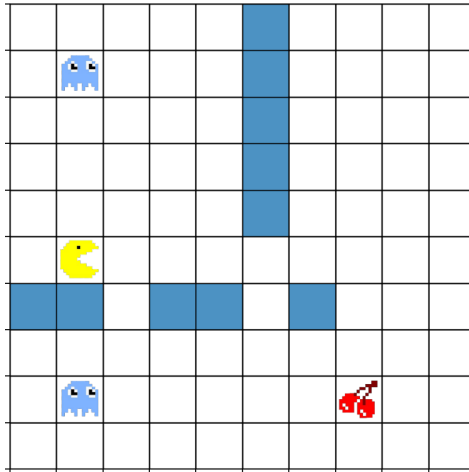


Figure 1: Initial 10×10 grid (static): Pacman start, dot, walls, and two ghosts.

3 Exploration Policy: ε -Greedy

In order to learn a policy and find the optimal behavior (avoiding converging too soon), an ε -greedy exploration is adopted, selecting a random action with probability ε_t and the greedy action otherwise. Two schedules are considered (based on Exploration-Exploitation using Exponentiated Gradient):

$$\text{Linear: } \varepsilon_t = \varepsilon_{\min} + (\varepsilon_0 - \varepsilon_{\min})\left(1 - \frac{t}{T}\right), \quad \text{Exponential: } \varepsilon_t = \varepsilon_{\min} + (\varepsilon_0 - \varepsilon_{\min})e^{-t/\tau}.$$

Unless stated otherwise, an exponential schedule in code for DQN, Q-learning, and SARSA is employed.

4 Tabular Methods: Q-learning vs. SARSA

Bellman's equation. All Q-learning-based-algorithms are formulated from Bellman's equation:

$$\underbrace{Q^*(s, a)}_{\text{optimal state-action value}} = \mathbb{E} \left[\underbrace{r + \underbrace{\gamma \max_{a'} Q^*(s', a')}_{\text{best future value}}}_{\text{expected return following the optimal policy}} \mid s, a \right].$$

discount factor

Iterative Algorithms. Q-learning basic implementation is off-policy and updates towards the greedy next-action value:

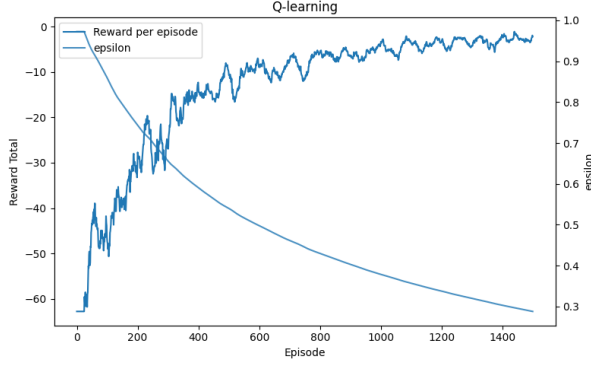
$$Q(s, a) \leftarrow Q(s, a) + \underbrace{\alpha}_{\text{learning rate}} \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

SARSA (State-Action-Reward-State-Action) is on-policy and updates toward the value of the next behavior action:

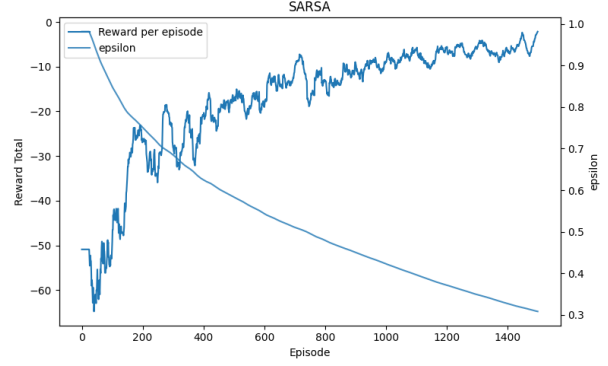
$$Q(s, a) \leftarrow Q(s, a) + \underbrace{\alpha}_{\text{learning rate}} \left[r + \gamma Q(s', a') - Q(s, a) \right].$$

Training curves. Each episode, the total reward during training is stored. The evolution of ε -greedy parameter is also displayed. A moving average method is used to smooth the curves and will also be applied in the next plot, allowing an easier interpretation. For this first test, $\alpha = 0.5$ and $\gamma = 0.99$.

With the previous configuration grid, maximum total reward possible equals to 2. Indeed, by knowing the optimal path, it is trivial to calculate it. As expected in a deterministic environment, those algorithms converged towards the optimal Q-table, finding the optimal behaviors after relatively few training episodes. However, SARSA algorithms seem to converge slower. But it is due to its on-policy nature, where updates are based on the actual exploratory actions taken rather than the optimal ones assumed by basic Q-learning.



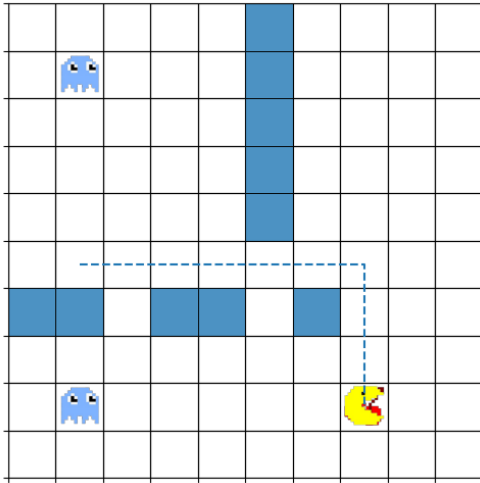
(a) Q-learning: Total reward per episode.



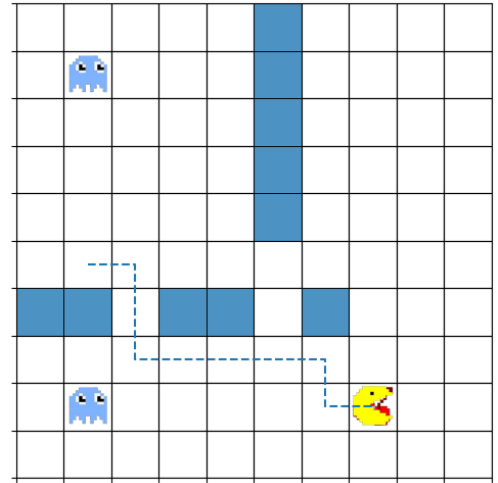
(b) SARSA: Total reward per episode.

Figure 2: Comparison of learning curves for Q-learning and SARSA on the static 10×10 grid.

Non unicity. In addition, the optimal path is not necessary unique and, hence, depends on the training.



(a) Q-learning: Behavior after training.

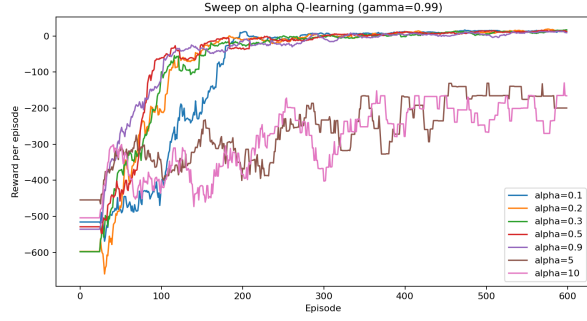


(b) SARSA: Behavior after training.

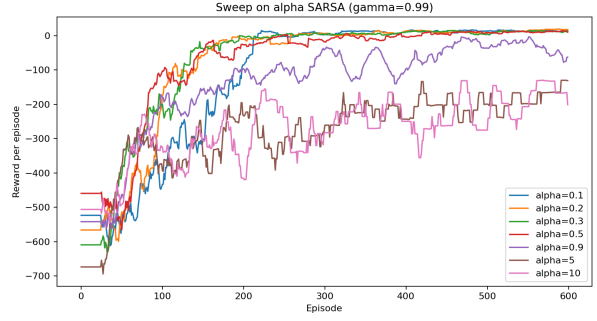
Figure 3: Comparison of behavior after training for Q-learning and SARSA on the static 10×10 grid.

Impact of α and γ . Find optimal set of parameters in machine learning is known as an NP-Hard problem. Here, vary one hyperparameter at a time while keeping others constant will be sufficient to study their influence. Then, the Total reward per episode curve is plotted for restricted set of parameters.

Parameter α . α is the learning rate and influences the convergence speed during training, more precisely how much the new information influences the old.



(a) Q-learning: Total reward per episode for different value of α .



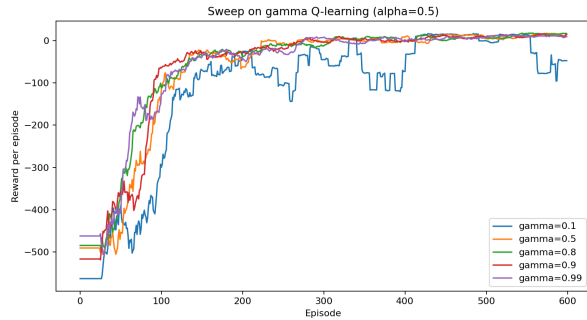
(b) SARSA: Total reward per episode for different value of α .

Figure 4: Influence of α during training for Q-learning and SARSA on the static 10×10 grid.

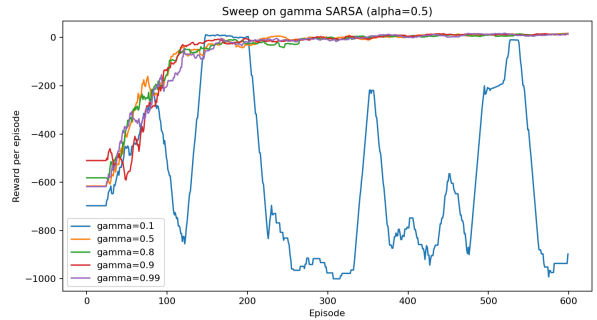
As expected for off-policy Q-learning algorithm, learning is directly proportional to the slope of the curves, mostly at the beginning of the training. So, use α near to 1 might be ideal in a static environment. But noticing the inverse tendency for SARSA could seem contradictory. Nonetheless, in this on-policy algorithm, the update target is stochastic. Therefore, larger values considerably reduces stability. A intermediate value (around 0.5) for α might be ideal.

Furthermore, the curves for $\alpha = 5, 10$ are anecdotal. As a matter of fact, Watkins and Dayan (1992) have demonstrated that $\alpha \in]0, 1[$ is a condition of convergence. Intuitively, it is possible to visualize it by realizing that $\alpha > 1$ tends to increase the error between two episodes, and this results in oscillations during training.

Parameter γ . γ is the discount factor and controls how much future rewards influence current decisions, more precisely how far the agent looks ahead when evaluating actions.



(a) Q-learning: Total reward per episode for different value of γ .



(b) SARSA: Total reward per episode for different value of γ .

Figure 5: Influence of γ during training for Q-learning and SARSA on the static 10×10 grid.

When γ increases, the agent values future rewards more, promoting long-term planning and, hence, smoother convergence. In contrast, when γ is close to zero, the agent becomes "myopic", focusing mainly on immediate rewards. In this case, value propagation between states fades quickly, and the learned policy fails to capture long-term dependencies. This effect is more pronounced for SARSA, whose on-policy updates make it sensitive to local reward variations during training.

5 Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search (MCTS) is an intelligent exploration algorithm. Unlike the previous learning-based methods, it does not learn from past experience. Its simple principle consists in randomly exploring possible trajectories in the environment and selecting the behavior that yields the highest total reward.

Principle and Parameters. Monte Carlo Tree Search (MCTS) is a tree-based planning method that tries to find the best action by simulating many possible futures. It borrows its main idea from the Upper Confidence Bound (UCB) algorithm, using a similar principle to balance exploration and exploitation when building the search tree.

The main parameters of MCTS are:

- c : controls the trade-off between exploration and exploitation. Higher values encourage trying less-visited actions, while lower ones make the algorithm focus more on exploiting the best-known ones.
- γ : discount factor used to weight future rewards during simulations.
- Number of simulations : defines how many trajectories are tested (more simulations usually give better accuracy but are slower).
- Rollout depth : sets how far the simulated actions can go in each test.

Choice of initial parameters. The following experiments show how these parameters affect Pacman’s decision-making and performance. For better interpretation, $rollout = 20$ and $\gamma = 0.99$. It is important to note that when the dot lies farther than the chosen rollout horizon, MCTS cannot directly observe its reward during simulations, which may lead to more stochastic or locally optimal behaviors. This limitation can be mitigated when $rollout \geq$ minimum step required to find the optimal behavior.

As the number of simulations just impacts the precision as the number of episodes, it won’t be mentioned after. A comment on the time complexity will be done later.

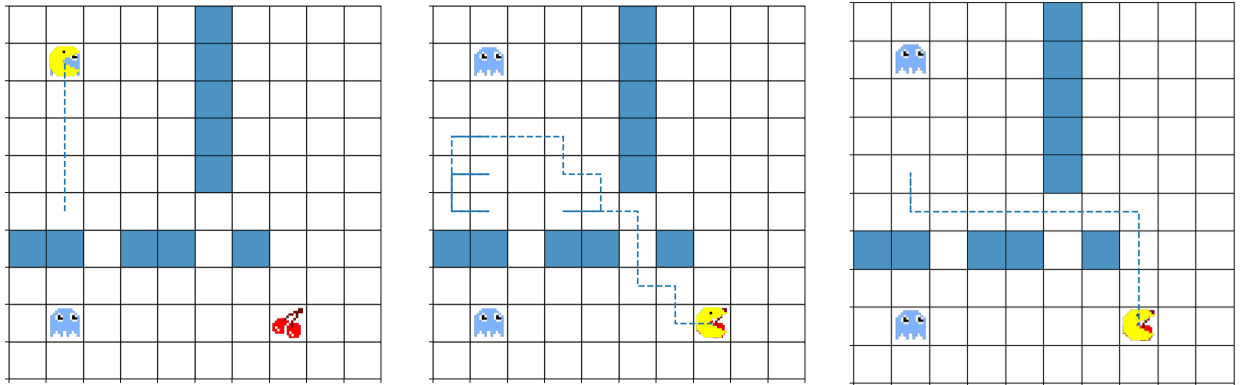
The exploration parameter is set to $c = 1$, a common value in simple environments. Since the system dynamics are limited and the search space relatively small, tuning c is not expected to bring significant improvement at first glance. To compare, using either a linear or an exponential $\epsilon - greedy$ schedule in the previous algorithms produces nearly identical results in this environment, indicating that the choice of schedule has little influence on convergence.

Remark on rollout depth. The rollout depth parameter does not directly limit the total number of steps in Pacman’s trajectory. It only defines how far each internal simulation (used to estimate action values) goes during the planning phase. At each real step, MCTS selects one action based on multiple simulated rollouts and then restarts the process from the next state. Therefore, even with a small rollout depth, the final path found by MCTS can be much longer than the simulation horizon.

Behaviors and Interpretation. With the current configuration, Step cost $c_{\text{step}} = -1$, dot reward $r_{\text{dot}} = +10$, ghost penalty $r_{\text{ghost}} = -10$, MCTS can't find the optimal behavior. It is a mathematical problem due to the bad choices of reward values.

In this configuration, the dot is located much farther from Pacman than the ghosts (8 cells away versus 4). Since MCTS does not learn value functions, it relies only on the outcomes of its random rollouts. During early simulations, most of the explored trajectories end quickly by colliding with a ghost, producing moderately negative rewards. Because each step has a cost of $r_{\text{step}} = -1$, taking long trajectories without reaching the dot accumulates heavy penalties. Consequently, it becomes numerically more advantageous for the algorithm to reach a ghost rapidly and end the episode rather than keep losing reward over many steps. As a result, the algorithm tends to favor these shorter trajectories rather than exploring longer ones leading to the dot, which remain underrepresented in the tree. Technically, an optimal path exists and would yield a positive return, but discovering it would require significant exploration at the beginning, which rarely happens in practice under limited simulation budgets.

Solution. The solution is not unique. However, it relies on choosing a balanced set of parameter for γ , r_{step} , r_{dot} and r_{ghost} . A first example is to increase considerably the r_{ghost} . In this case, MCTS would rather search other directions than converge to the ghost. However, the optimal path is not yet found even if pacman find the dot at the end. Increasing r_{step} could be a good idea. Finally, reducing γ could lead to reduce impact of step penalty but will also reduce impact of the dot. So, it must be tuned accordingly to the dot reward.



(a) $r_{\text{ghost}} = -10$, $r_{\text{dot}} = +10$, $r_{\text{step}} = -1$, (b) $r_{\text{ghost}} = -100$, $r_{\text{dot}} = +10$, $r_{\text{step}} = -1$, (c) $r_{\text{ghost}} = -1000$, $r_{\text{dot}} = +100$, $r_{\text{step}} = -10$

Figure 6: Different behaviors found by MCTS under various reward configurations. Increasing the ghost penalty and dot reward encourages safer and more goal-oriented trajectories.

6 Deep Q-Network (DQN)

Principle. The Deep Q-Network (DQN) extends the traditional Q-learning algorithm by replacing the discrete Q-table with a neural network that estimates the action-value function $Q_{\theta}(s, a)$. Given the current state as input, the network predicts the expected return for each possible action, allowing the agent to generalize across similar situations. We approximate $Q_{\theta}(s, a)$ with a small convolutional

network on the (C, H, W) grid, trained using an experience replay buffer and a target network to stabilize learning. The network parameters are optimized by minimizing the Bellman loss:

$$L(\theta) = (Q_{\theta}(s, a) - [r + \gamma \max_{a'} Q_{\theta-}(s', a')])^2.$$

This implementation follows the standard PyTorch-style training loop, with a simplified architecture adapted to the Pacman environment.

Network choice. A very small architecture is used. It is suitable for 10×10 inputs and consists in a single convolutional block followed by a 128-unit fully connected (FC) head. This lightweight structure is sufficient to capture spatial dependencies in the grid while keeping computation fast and stable. Larger architectures were tested but did not yield significant improvements due to the simplicity of the environment. The convolutional layer extracts local spatial features, and the FC layer maps them to action values $Q(s, a)$ for each possible move.

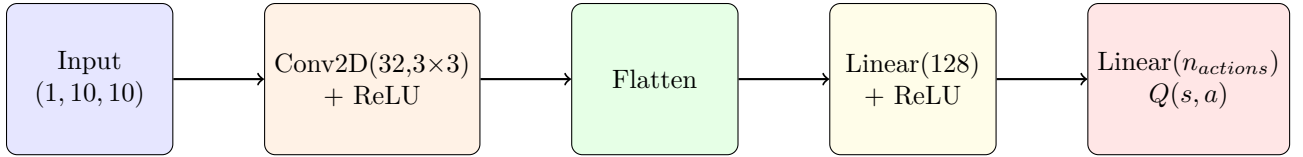
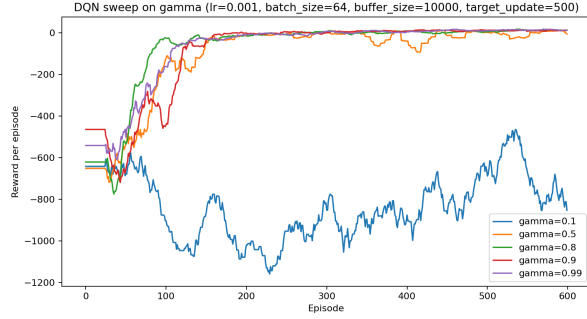


Figure 7: Simplified DQN architecture used for the Pacman grid environment (10×10). The model remains intentionally small to avoid overfitting and ensure fast convergence.

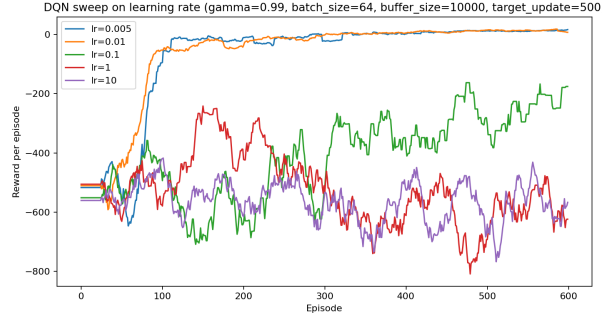


Figure 8: Comparison of total reward per episode for Q-learning, SARSA, and DQN on the static 10×10 grid.

Training curves comparison. As expected, DQN converges towards the optimal behavior even when using a simple structure of layers. However, at first glance, it has no difference with basic Q-learning algorithm for the deterministic environment.



(a) DQN: Total reward per episode for different values of γ .



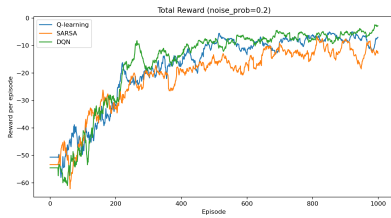
(b) DQN: Total reward per episode for different learning rates.

Figure 9: Influence of γ and learning rate during DQN training on the static 10×10 grid.

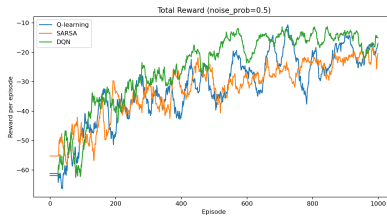
Influence of Parameter lr (learning rate) and γ . Without surprise, the observations is very similar to the Q-learning algorithm. Small γ brings divergence and Large learning rate produces oscillations during training. However, the learning rate for DQN is scaled differently as very low values are needed for convergence.

7 Robustness to Noise

Setup. Robustness is measured by injecting action noise with probability p_n at each step. An analysis of the training rewards for tabular and DQN methods is realized and win-rate becomes informative here (previously 100% in static settings).



(a) $p_n = 0.1$



(b) $p_n = 0.5$



(c) $p_n = 1.0$

Figure 10: Total reward per episode for Q-learning, SARSA, and DQN under increasing noise probabilities p_n . Each curve is smoothed using a moving average for better readability.

Table 1: Win-rate (%) for each method under increasing noise probabilities p_n .

Method	$p_n = 0.1$	$p_n = 0.5$	$p_n = 1.0$
Q-learning	100	79	8
SARSA	100	69	5
DQN	100	89	10
MCTS	100	96	4

Results and Interpretation. As expected, all methods maintain perfect performance when the noise level remains low ($p_n = 0.1$), demonstrating good stability in near-deterministic conditions. However, as the noise increases, the tabular methods (Q-learning and SARSA) degrade rapidly, since their policies directly depend on the accuracy of state transitions. MCTS shows a slower degradation thanks to its planning nature and seems to outperform DQN when $p_n = 0.5$. Nevertheless, this apparent improvement is misleading: in practice, MCTS remains far more stochastic and less stable than DQN, as will be observed later in the study conducted under a stochastic environment.

8 Stochastic Environment

Setup. The stochastic environment is evaluated by enabling random starting positions and varying the ghost movement probability p_g . Higher p_g values make the environment increasingly unpredictable, forcing the agent to adapt to dynamic opponents and non-deterministic transitions.

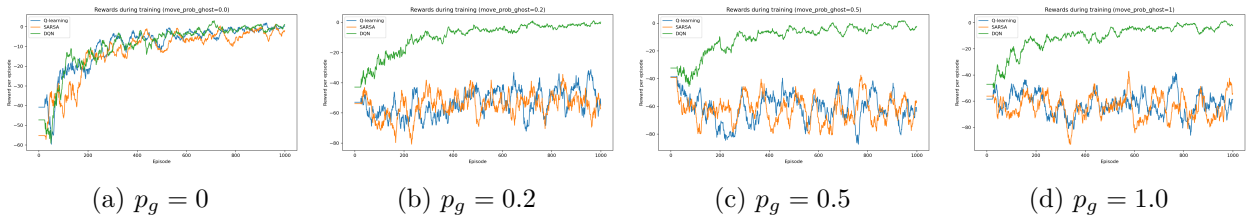


Figure 11: Total reward per episode for Q-learning, SARSA, and DQN under increasing ghost movement probabilities p_g . The first configuration ($p_g = 0$) represents the case where only random starting positions are enabled. Each curve is smoothed using a moving average for better readability.

Table 2: Win-rate (%) for each method under increasing ghost movement probabilities p_g . The first configuration ($p_g = 0$) corresponds to random starting positions only.

Method	$p_g = 0.0$	$p_g = 0.2$	$p_g = 0.5$	$p_g = 1.0$
Q-learning	100	79	16	4
SARSA	100	69	12	10
DQN	100	100	98	97
MCTS	100	86	86	76

Results and Interpretation. As the ghost movement probability p_g increases, the environment becomes progressively more dynamic and harder to predict. Both tabular methods, Q-learning and SARSA, show a strong degradation in performance, struggling to adapt to changing ghost behaviors and random starts. MCTS performs more consistently thanks to its simulation-based planning, but remains limited by its lack of learning and the stochastic nature of rollouts. In contrast, DQN demonstrates remarkable robustness, maintaining almost perfect win-rates even in highly dynamic conditions ($p_g = 1.0$). A deeper hyperparameter study under these stochastic conditions could slightly alter the results, but the overall trend would likely remain the same in such a simple environment.

Although MCTS sometimes achieves relatively high win-rate values compared to DQN, the number of steps required to reach the dot is significantly higher, resulting in a much lower mean reward.

Moreover, the reward were manually adjusted to allow MCTS to perform reasonably well. Without such tuning, its results would be substantially worse.

9 Global Performance Overview

We summarize qualitative advantages and disadvantages using a simple $-/+$ scale (more $+$ is better).

Table 3: Qualitative comparison of the four methods ($-$ poor, $+$ good, $++$ very good, $+++$ excellent).

Method	Static env.	Stochastic env.	Randomness	Noise sens.	Training time	Inference / Search time	Memory
Q-learning	+++	$-$	Low	High	++	+++	+
SARSA	+++	$-$	Medium	High	++	+++	+
MCTS	++	++	High	Low	None	$-$	+++
DQN	+++	+++	Medium	Low	$-$	++	$-$

10 Conclusion

This report presented a comparative study of several reinforcement learning and planning methods applied to a simplified Pacman grid environment. Starting from tabular algorithms such as Q-learning and SARSA, the analysis progressively introduced more advanced approaches like MCTS and DQN. Each method demonstrated its own strengths and weaknesses depending on the environment conditions.

In deterministic scenarios, tabular methods quickly converged to optimal behaviors, while DQN achieved similar performance with better robustness to noise and random dynamics. MCTS, although conceptually elegant and efficient in small environments, proved less stable and more computationally demanding in practice. Moreover, its performance was shown to be highly dependent on the specific choice of reward values, much more than in the other methods.

Beyond raw performance, this study highlighted the sensitivity of learning parameters such as α and γ , as well as the importance of balancing exploration and exploitation through ε -greedy schedules. Overall, the results confirm that even in a simple game like Pacman, designing an effective reinforcement learning agent requires both theoretical understanding and careful empirical tuning.

Future work could extend this analysis to larger grids, additional ghosts, or continuous environments, in order to better understand how these algorithms scale and generalize in more complex situations.

References and Context

This report is mainly based on course materials and original implementation. Key references include: Watkins & Dayan (1992) for Q-learning, Vodopivec et al. (2017) for MCTS, Mnih et al. (2015) for DQN, and the official PyTorch tutorial on reinforcement Q-learning (https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html).